



Universidad de

los Andes



**FACULTAD
DE INGENIERÍA
Y CIENCIAS
APLICADAS**

Arquitectura de computadores

Diseño Lógico y FPGA

Integrantes

Agustín Álvarez

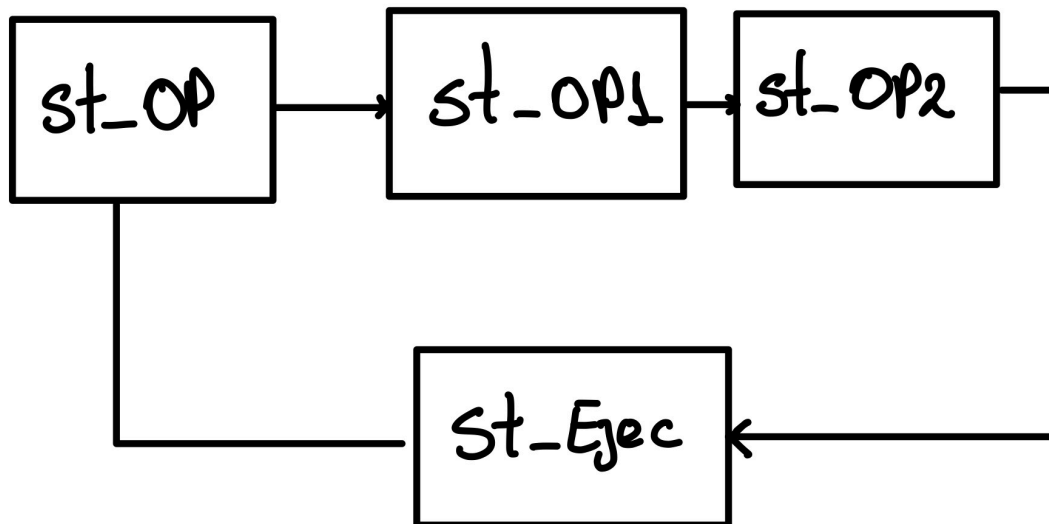
Arantxa Toledo

Samuel Vallete

Profesor

Jorge Gómez Mir

1. Diseño general y arquitectura de la calculadora



ST_OP: Muestra los operandos

ST_OP1: aumenta o disminuye el número hasta escoger uno.

ST_OP2: se puede hacer lo mismo que en ST_OP1 o se puede usar el resultado anterior.

ST_EJEC: después de mostrar el resultado en la pantalla se apreta el boton y se reinicia el proceso

2. Tablas de verdad, mapas de Karnaugh y expresiones booleanas.

1. Half adder

La función Half Adder es utilizada en la función bit4_adder (suma de cuatro bits), al sumar el primer bit de cada número ingresado y en consecuencia, es aplicado indirectamente en bit4_sub (resta) y bit4_inv_sub (resta inversa), ya que la resta es solo una suma común con números binarios negativos.

Tabla de verdad:

A	B	Carry	Suma
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Mapa de Karnaugh:

Para Carry:

A	B	0	1
0		0	0
1		0	1

Para Suma:

A	B	0	1
0		0	1
1		1	0

Expresiones booleanas:

Las expresiones booleanas del half_adder se resumen en dos fórmulas:

$$\text{Carry} = A \text{ AND } B$$

$$\text{Suma} = A \text{ XOR } B$$

2. Full adder

La función Half Adder es utilizada en la función bit4_adder (suma de cuatro bits), al sumar del segundo al cuarto bit de cada número ingresado y en consecuencia (al igual que half_adder), es aplicado indirectamente en bit4_sub (resta) y bit4_inv_sub (resta inversa).

Tabla de verdad:

A	B	Carry In	Carry Out	Suma
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Mapa de Karnaugh:

Para Carry Out:

A B	Carry In	0	1
0 0		0	0
0 1		0	1
1 0		0	1
1 1		1	1

Para Suma:

A B	B	0	1
0 0		0	1
0 1		1	0
1 0		1	0
1 1		0	1

Expresiones booleanas:

Las expresiones booleanas del half_adder se resumen en dos fórmulas:

$\text{CarryOut} = (A \text{ AND } B) \text{ OR } (\text{CarryIn AND } (A \text{ XOR } B))$

$\text{Suma} = A \text{ XOR } B \text{ XOR } \text{CarryIn}$

3. Shift left

Dado una entrada A, un bit de movimiento B y una salida C, en la siguiente tabla de verdad se expresa una posible salida de acuerdo a los valores de B. La expresión A[n] significa que el valor de ese bit de salida depende exclusivamente del valor del bit de A en esa posición.

Tabla de verdad:

B[1]	B[0]	C[3]	C[2]	C[1]	C[0]
0	0	A[3]	A[3]	A[1]	A[0]
0	1	A[2]	A[1]	A[0]	0
1	0	A[1]	A[0]	0	0
1	1	A[0]	0	0	0

Mapa de Karnaugh:

El mapa de Karnaugh se presentará para los primeros dos dígitos de salida, ya que debido a la extensión de esta tabla, es infactible incluirla para el tercer y cuarto dígito.

Para C[0]

B[1] B[0] A[0]	0 0	0 1	1 1	1 0
0	0	0	0	0
1	1	0	0	0

Para C[1]

B[1] B[0] A[0]	0 0	0 1	1 1	1 0
0 0	0	0	0	0
0 1	0	1	0	0
1 0	1	0	0	0
1 1	1	1	0	0

Expresiones booleanas:

C[0]:

$$C[0] = A[0] \text{ AND NOT } B[1] \text{ AND NOT } B[0]$$

C[1]:

$$C[1] = (A[1] \text{ AND NOT } B[1] \text{ AND NOT } B[0]) \text{ OR } (A[0] \text{ AND NOT } B[1] \text{ AND } B[0])$$

C[2]:

$$C[2] = (A[2] \text{ AND NOT } B[1] \text{ AND NOT } B[0]) \text{ OR } (A[1] \text{ AND NOT } B[1] \text{ AND } B[0]) \text{ OR } (A[0] \text{ AND } B[1] \text{ AND NOT } B[0])$$

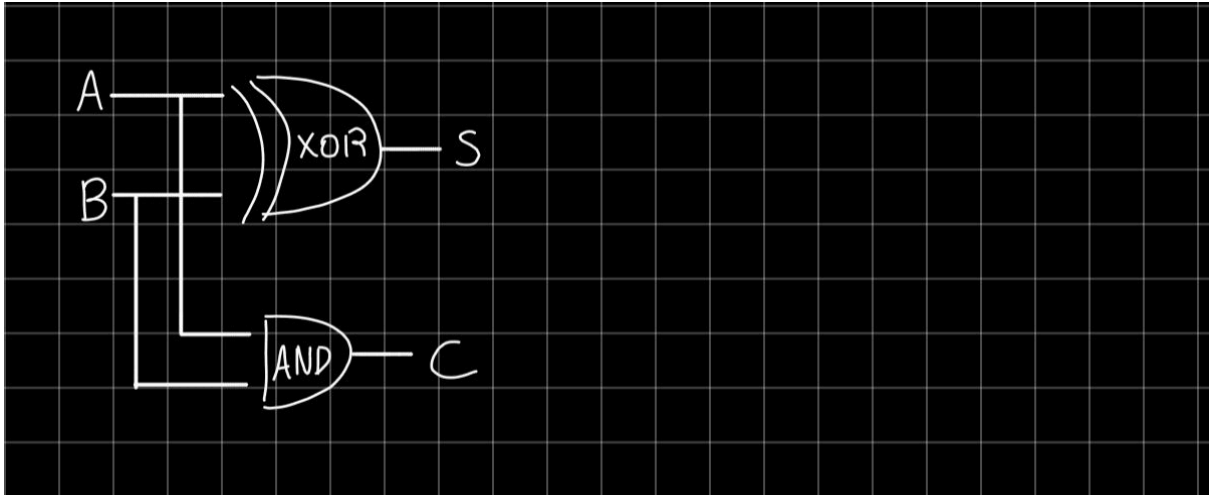
C[3]:

$$C[3] = (A[3] \text{ AND NOT } B[1] \text{ AND NOT } B[0]) \text{ OR } (A[2] \text{ AND NOT } B[1] \text{ AND } B[0]) \text{ OR } (A[1] \text{ AND } B[1] \text{ AND NOT } B[0]) \text{ OR } (A[0] \text{ AND } B[1] \text{ AND } B[0])$$

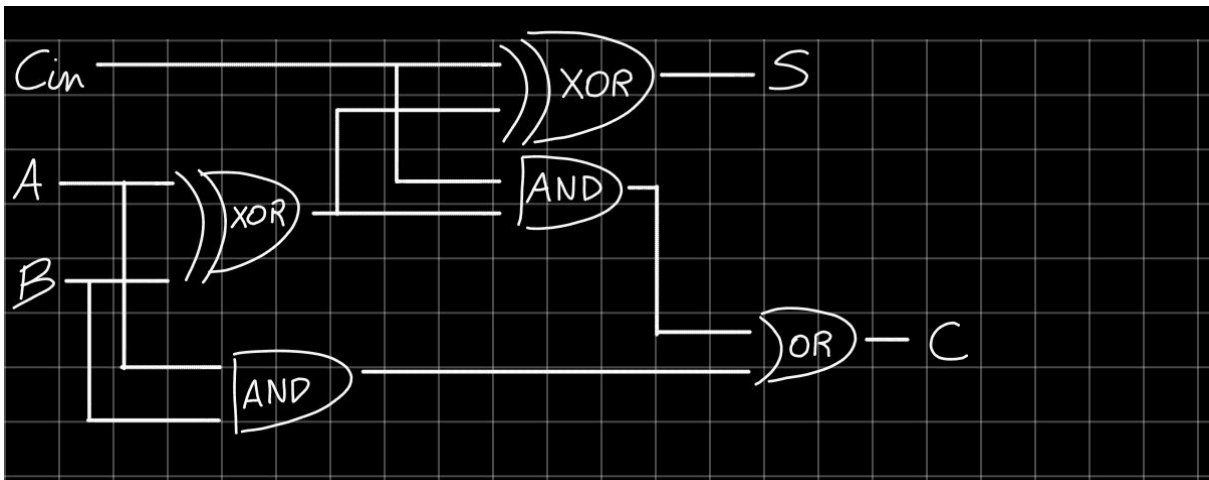
3. Implementación de las operaciones mediante compuertas

En el siguiente ítem se mostrarán los diagramas de compuertas de cada una de las funciones y operaciones.

1. Half adder

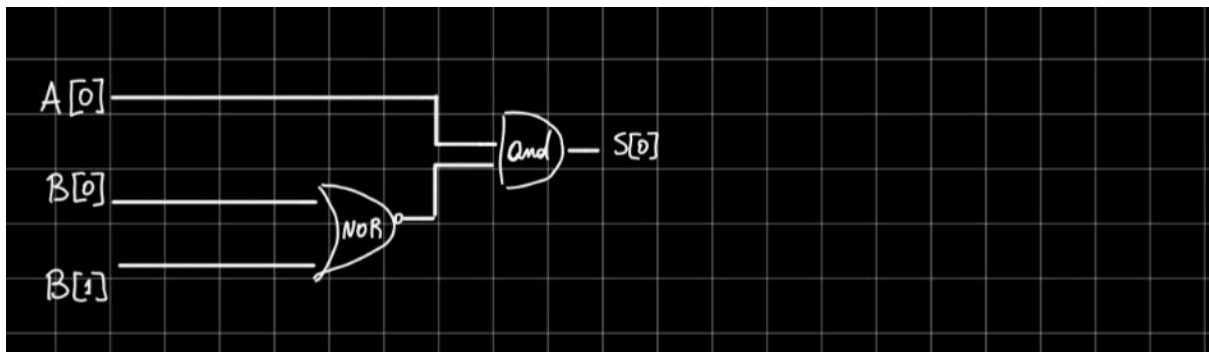


2. Full adder

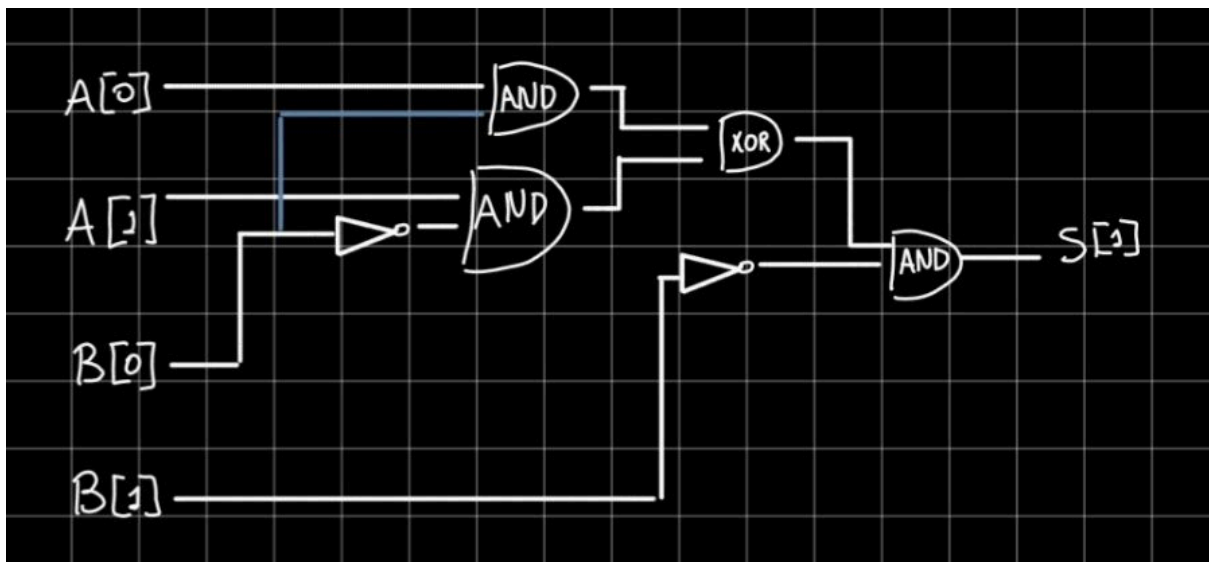


3. Switch left

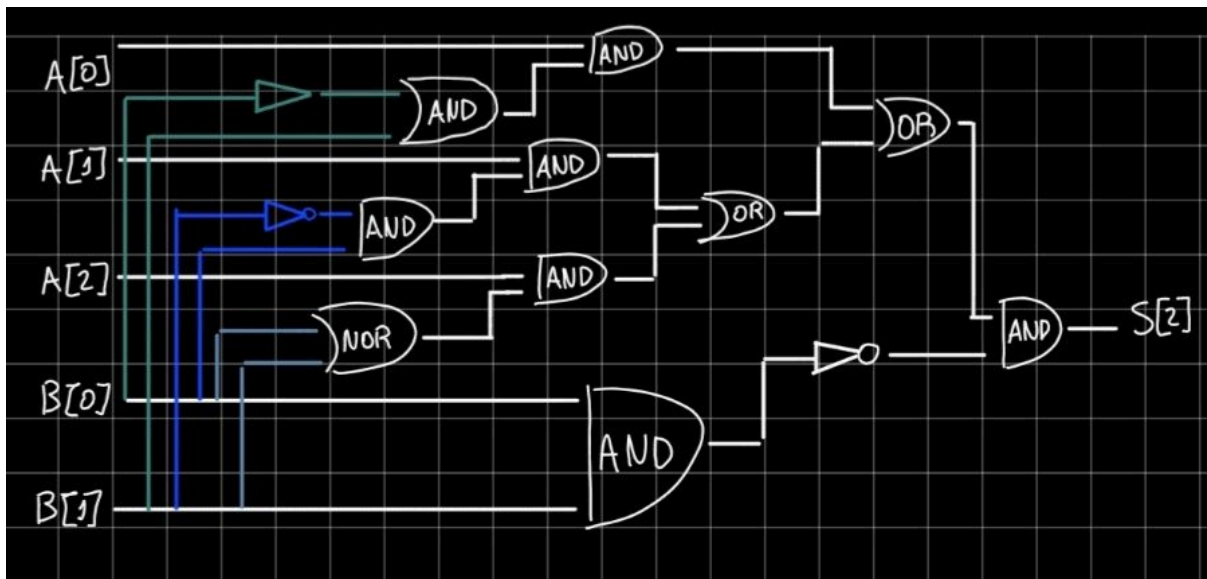
3.1 Primer dígito del switch left



3.2 Segundo dígito del switch left



3.3 Tercer dígito del switch left



3.4 Cuarto dígito del switch left

